

结构的魔力

基础数据结构 · 8题 · C++ & Python 双语对照

小鲁已经学会了不少算法技巧——排序、二分、前缀和、双指针。但小华告诉他："真正的程序性能瓶颈往往不在算法，而在**数据结构**。选对数据结构——你的代码可以快100倍。栈来处理后进先出，队列来处理先进先出，单调栈来找下一个更大元素，滑动窗口用单调队列，字符串匹配用KMP，优先队列用堆.....本章8道题，每一题都是一个数据结构选择的经典案例。

本章角色

-  **小鲁** (进阶者) · 从数组迈入抽象数据结构——栈/队列/堆/KMP
-  **小华** (算法队长) · 白板上画出KMP的next数组跳转、堆的up/down操作
-  **小栋** (工程师) · "单调队列求滑动窗口最值"——真实大数据场景的最优解

八大基础数据结构速查

- **单链表**: 数组模拟——e[],ne[],head,idx。全部操作O(1)
- **栈/队列**: 数组+指针。Python: list=栈, deque=队列
- **单调栈**: 维护单调性的栈——找左右第一个更小/大值。均摊O(n)
- **单调队列**: 维护单调性的双端队列——滑动窗口最值。O(n)
- **KMP**: next数组+失配跳转——字符串匹配O(n+m)
- **堆**: 完全二叉树+up/down——优先队列O(log n)插入删除

NQ127 单链表 AcWing 826

小鲁第一次实现链表："每个节点有一个next指针指向下一个节点。但反复new/delete太慢了——竞赛中用的是数组模拟链表！"小华解释道："e[i]存节点i的值，ne[i]存节点i的下一个节点下标。头节点head指向第一个节点，空闲链表管理未用节点。三个数组+一个整数idx（当前可用下标）就搭建了完整的内存池。这就是为什么Oler的代码里满是数组——不是不会用指针，而是数组更快。"

输入 第一行M。然后M行操作：H x(头插)、D k(删第k后)、I k x(在第k后插入)。

输出 链表从头到尾的值。

范围 $1 \leq M \leq 100000$

输入

```
10
H 9
I 1 1
D 1
D 0
H 6
I 3 6
```

输出

```
6 4 6 5
```

```
I 4 5
I 4 5
I 3 4
D 6
```

思路 数组模拟链表：e[]存值，ne[]存next下标，head=头节点下标，idx=当前可用下标。头插法：e[idx]=x; ne[idx]=head; head=idx++。第k个后插入：e[idx]=x; ne[idx]=ne[k]; ne[k]=idx++。删除第k个之后：ne[k]=ne[ne[k]]。

技巧 数组模拟链表=静态链表。每个操作O(1)。Python用list同样高效（append/pop=O(1)）。理解这种'用数组下标替代指针'的思想——后续Trie、并查集、堆都是用数组模拟各种结构。

C++

```

#include <iostream>

using namespace std;

const int N = 100010;

// head 表示头节点的下标, e[N]表示节点i的值
// ne[N]表示节点i的next指针是多少, idx存储当前已经用到了哪个节点
int head, e[N], ne[N], idx;

void init()
{
    head = -1; // 一开始设置为-1
    idx = 0; // 当前用到了0个节点
}

// 将x插入到头节点
void add_to_head(int x)
{
    e[idx] = x, ne[idx] = head, head = idx, idx++;
}

// 将x插到下标是k的点后面
void add(int k, int x)
{
    e[idx] = x, ne[idx] = ne[k], ne[k] = idx, idx++;
}

// 将下标是k的点后面的点删掉
void remove(int k)
{
    ne[k] = ne[ne[k]];
    // idx--;
}

int main()
{
    int m; cin >> m;

    init(); // 初始化

    while (m--)
    {
        int k, x;
        char op; // 操作字符

        cin >> op;
        if (op == 'H')
        {
            cin >> x;
            add_to_head(x);
        }
        else if (op == 'D')
        {
            cin >> k;
            if (!k) head = ne[head]; // 删除头节点
            else remove(k-1);
        }
        else
        {
            cin >> k >> x;
            add(k-1, x);
        }
    }

    // 打印单链表
    for (int i = head; i != -1; i = ne[i]) cout << e[i] << '

```

Python

Python solution

```
;
cout << endl;
}
```

NQ128 模拟栈 AcWing 828

"栈——后进先出(LIFO)。就像叠盘子，最后放上去的最先拿下来。"小华说，"实现一个栈支持push和pop操作。C++有std::stack，Python用list的append()和pop()天然就是栈。但手写一个数组模拟理解底层更透彻——用一个数组stk和指针tt，push时stk[++tt]=x，pop时tt--只移动指针，O(1)时间。"

输入 第一行M。然后M行：push x / pop / empty / query。

输出 按操作输出。

范围 $1 \leq M \leq 100000$

输入	输出
6	5
push 5	5
query	NO
push 6	
pop	
query	
empty	

思路 数组模拟栈：stk[N]和tt（栈顶指针，初始0）。push: stk[++tt]=x；pop: tt--；query: stk[tt]；empty: tt>0?'NO':'YES'。指针只是整数下标，极其高效。Python: list.append()=push, list.pop()=pop, list[-1]=query。

技巧 单调栈就是在普通栈的基础上加一条约束——栈内元素保持单调。这是后续处理'下一个更大元素'等问题的核心数据结构。理解栈+单调性=>O(n)解决看似O(n²)的问题。

C++

```
#include <iostream>

using namespace std;
const int N = 100010;

int m;
int stk[N], tt; // 数组模拟栈

int main()
{
    cin >> m;
    while (m --)
    {
        string op;
        int x;

        cin >> op;
        if (op == "push") // 栈顶指针+1, 把数压入数组
        {
            cin >> x;
            stk[++tt] = x; // 压入一个元素
        }
        else if (op == "pop") tt --; // 修改栈顶指针即可
        else if (op == "empty") cout << (tt? "NO": "YES") << endl; // tt>0表示非空
        else cout << stk[tt] << endl; // 返回栈顶
    }
}
```

Python

Python solution

NQ129 模拟队列 AcWing 829

"队列——先进先出(FIFO)。就像排队买饭，先到的先服务。"小华画出示意图，"用数组模拟队列需要两个指针：hh(队头)和tt(队尾)。push时q[++tt]=x，pop时hh++——两个指针都只进不退。这就是'同时移动双端'的模式。Python的deque是双端队列，from collections import deque——比list的pop(0)快(O(1) vs O(n))。"

输入 第一行M。然后M行：push x / pop / empty / query。

输出 按操作输出。

范围 $1 \leq M \leq 100000$

输入	输出
6	5
push 5	6
query	NO
push 6	
pop	
query	
empty	

思路 数组模拟队列：q[N]，hh=0队头，tt=-1队尾。push: q[++tt]=x; pop: hh++; query: q[hh]; empty: hh>tt。注意hh>tt表示空。Python: deque.append()=push, deque.popleft()=pop。list的pop(0)是O(n)——

不要用！

技巧 Python中list.pop(0)是O(n)因为要移动所有元素。一定要用collections.deque！单调队列=单调性+双端队列——滑动窗口最值问题的核心数据结构。

C++

```
#include <iostream>
using namespace std;
const int N = 1000007;
int q[N], hh, tt=-1;

int main(){
    int m;
    cin>>m;
    while (m--){
        int x;

        string op;
        cin>>op;

        if (op == "push") {
            //(1) "push x" - 向队尾插入一个数x;
            cin>>x;
            q[++tt] = x;
        } else if (op == "pop") //(2) "pop" - 从队头弹出一个数;
            hh++;
        else if (op == "empty") //(3) "empty" - 判断队列是否为空;
            cout<<(hh<=tt?"NO":"YES")<<endl;
        else cout<<q[tt]<<endl; //(4) "query" - 查询队头元素。
    }

    return 0;
}
```

Python

Python solution

NQ130 表达式求值 AcWing 3302

"计算'(2+3)×5'——用栈！"小华说，"两个栈：一个存数字，一个存运算符。遇到数字压数字栈；遇到(压运算符栈；遇到)计算直到(被弹出；遇到运算符——如果栈顶优先级 $\geq$ 当前，先算栈顶。核心：维护运算优先级。这是编译原理和计算器的核心。"小鲁试了试："确实巧妙——中缀表达式转后缀求值的经典算法！"

输入 一个中缀表达式（含+、*、/和括号，数字非负）。

输出 表达式结果（整数除法向零截断）。

范围 表达式长度 ≤ 100000

输入	输出
(2+3)*5	25

思路 双栈法求中缀表达式：数字栈+运算符栈。优先级映射：+-为1，*/为2。遇到)弹栈直到(。每次弹出一个运算符和两个数字，计算结果压回数字栈。Python的eval()能直接算但OJ比赛中禁止。理解手写栈求值是数

据结构综合应用的经典案例。

技巧 符号优先级存入字典：`pri={'+':1,'-':1,'*':2,'/':2}`。从左到右扫描。`eval()`在安全环境下可用但不适用于OJ——学会手写才是目的。注意除法向零截断——Python的`//`对负数向下取整，需要用`int(a/b)`。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <stack>
#include <unordered_map>

using namespace std;

stack<int> num; //存储计算过程中的数字
stack<char> op; //存储操作符

void eval()
{
    auto b = num.top(); num.pop(); //第二个数
    auto a = num.top(); num.pop(); //第一个数
    auto c = op.top(); op.pop();

    int x;
    if (c == '+') x = a + b;
    else if (c == '-') x = a - b;
    else if (c == '*') x = a * b;
    else x = a / b;
    num.push(x);
}

int main()
{
    //优先级, 可扩展为其他符号的优先级
    unordered_map<char, int> pr{{'+', 1}, {'-', 1}, {'*', 2}, {'/', 2}};
    string str; //输入都存储在字符串内
    cin >> str;
    for (int i = 0; i < str.size(); i++)
    {
        auto c = str[i]; //读入每个char
        if (isdigit(c)) //判断是否是数字
        {
            int x = 0, j = i;
            while (j < str.size() && isdigit(str[j]))
                x = x * 10 + str[j++] - '0';
            i = j - 1; // 调整i的位置
            num.push(x); //压栈
        }
        else if (c == '(') op.push(c);
        else if (c == ')')
        {
            //把前面所有的表达式都出栈运算
            while (op.top() != '(') eval();
            op.pop();
        }
        else
        {
            //根据操作符的优先级执行运算
            while (op.size() && op.top() != '(' && pr[op.top()]
                >= pr[c])
            {
                eval();
                op.pop();
            }
            op.push(c);
        }
    }
    //处理剩余的操作符
    while(op.size()) eval();
    cout<<num.top() <<endl;
    return 0;
}

```

Python

Python solution

NQ131 单调栈 AcWing 830

"找每个数左边第一个比它小的数——暴力 $O(n^2)$ 太慢。"小华说，"单调栈——维护一个单调递增的栈。遍历每个数时，把栈中 $\geq$ 它的数都弹掉（因为它们不会再成为任何后续数的答案）。弹完之后栈顶就是答案，然后当前数入栈。每个数最多入栈一次出栈一次——均摊 $O(n)$ 。单调性把 $O(n^2)$ 降到 $O(n)$ ！"

输入 第一行N。第二行N个整数。

输出 每个数左边第一个比它小的数（没有输出-1）。

范围 $1 \leq N \leq 100000$

输入

5
3 4 2 7 5

输出

-1 3 -1 2 2

思路 维护单调递增栈：遍历每个数，while栈非空且栈顶 $\geq$ 当前数则pop()。弹出结束后栈顶就是答案。当前数入栈。每个数入栈1次出栈1次——均摊 $O(n)$ 。单调栈利用'大数再也不会被需要'的洞察来去掉无用信息。

技巧 单调栈的变种：找右边第一个大的（从右往左+单调递减），找左边第一个大的（从左到右+单调递减）。记模板不如理解核心：'弹出永远不会用到的元素'。Python: while stack and stack[-1] $\geq$ x: stack.pop()。

C++

```
#include <iostream>
using namespace std;
const int N=100007;
int n;
int stk[N],tt;//数组模拟栈实现单调栈
int main ()
{
    cin>> n;
    for (int i=0; i< n; i++){
        int x; cin>>x;
        while (tt && stk[tt]>= x)//tt为空，并且单调栈顶元素比
x大
            tt--;//出栈，这些数永远不会用到
        if (tt) cout<<stk[tt]<<' '; //栈顶就是比x小的元素，
直接输出
        else cout<<-1<<' '; //x左边没有任何数比它小
        //入栈
        stk[++tt] = x;
    }
    return 0;
}
```

Python

```
# Python solution
```

NQ132 滑动窗口 AcWing 154

"长度为k的窗口在数组上从左滑到右——输出每个窗口的最小值和最大值。"小栋说，"暴力每个窗口 $O(k)$ 总 $O(nk)$ 太慢。"小华回答："单调队列！用两端队列维护窗口内的最小值候选。队头是当前窗口的最小值下标，当

队头滑出窗口时弹出，当新元素比队尾更小时——队尾永无翻身之日，弹掉。 $O(n)$ 时间解决。这是单调队列最经典的应用！"

输入 第一行 n, k 。第二行 n 个整数。

输出 第一行所有窗口最小值。第二行所有窗口最大值。

范围 $1 \leq n \leq 10^6, 1 \leq k \leq n$

输入	输出
8 3	-1 -3 -3 -3 3 3
1 3 -1 -3 5 3 6 7	3 3 5 5 6 7

思路 单调队列维护窗口最值。最小值：维护递增队列，队头存窗口最小值下标。每次新元素 x 到来：while队尾值 $\geq x$ 则`pop_back`（队尾不会再成为最小值），然后`push_back`当前下标。if队头下标 $<$ 窗口左边界则`pop_front`。队头=当前窗口最小值。最大值类似（维护递减队列）。 $O(n)$ 。

技巧 Python用`collections.deque`—— $O(1)$ 的`popleft()`和`pop()`。队尾弹出while `q and arr[q[-1]]>=x`: `q.pop()`。理解单调队列的本质：维护一个'永远单调'的双端候选集。这是滑动窗口问题的终极答案。

C++

```

#include <iostream>
using namespace std;

const int N=10000007;

int n,k;
int a[N],q[N]; //数组模拟队列
int hh=0,tt=-1; //hh队头,tt对尾

void initQueue(){
    hh=0; tt=-1;
}

int main()
{
    scanf("%d%d",&n,&k);
    for (int i=0; i<n; i++) scanf("%d",&a[i]); //读入数字

    //求滑动窗口内的最小值
    initQueue(); //初始化队列, 队列存储的是数组下标
    for (int i=0; i<n; i++)
    {
        //判断队头是否已经滑出窗口
        if (hh<=tt && i-k+1> q[hh]) hh++; //pop front 删除队头

        //!单调队列, 去掉所有降序的队尾点
        while (hh<=tt&& a[q[tt]]>= a[i]) tt--; //pop back 删除队尾

        q[++tt] = i; //把当前i值入队列
        if (i>=k-1) //i在窗口内
            printf("%d ",a[q[hh]]); //队头就是最小值
    }
    puts("");

    //求滑动窗口内的最大值
    initQueue(); //初始化队列, 队列存储的是数组下标
    for (int i=0; i<n; i++)
    {
        //判断队头是否已经滑出窗口
        if (hh<=tt && i-k+1> q[hh]) hh++; //pop front 删除队头

        //!单调队列, 去掉所有降序的队尾点
        while (hh<=tt&& a[q[tt]]<=a[i]) tt--; //pop back 删除队尾

        q[++tt] = i; //把当前i值入队列
        if (i>=k-1) //i在窗口内
            printf("%d ",a[q[hh]]); //队头就是最大值
    }
    puts("");
}

```

Python

Python solution

NQ133 KMP字符串 AcWing 831

"字符串匹配——模式串P在文本串T中找所有出现位置。暴力匹配 $O(n \times m)$ 太慢。"小华打开白板，"KMP算法——预处理next数组，利用已经匹配的信息避免重复比较。next[i]=P[0..i-1]的最长公共前后缀长度。匹配时如果失配，j跳到next[j]——不需要回到最初的i+1。时间 $O(n+m)$ 。这是算法课上最优雅的设计之一。"

输入 第一行n和模式串P，第二行m和文本串T。

输出 所有匹配位置的起始下标（0-indexed）。

范围 $1 \leq n \leq 10^5, 1 \leq m \leq 10^6$

输入

3
aba
5
ababa

输出

0 2

思路 KMP两步骤：1)求next数组——i从1开始，j=next[i-1]；while j>0且P[i]!=P[j]则j=next[j-1]；若P[i]==P[j]则j++；next[i]=j。2)匹配——i遍历T,j跟踪P；while j>0且T[i]!=P[j]则j=next[j-1]；若T[i]==P[j]则j++；若j==n则找到匹配。Python的P in T或find()底层用高效算法但理解KMP原理是理解字符串算法的基石。

技巧 next数组=失配时跳转的'记忆'——利用模式串自身的前后缀重复避免回溯。KMP保证了每个字符最多被比较两次。Python: T.find(P)一行，但手写KMP理解'失配跳转'的智慧是进阶算法的必修课。

C++

```

#include <iostream>
using namespace std;
const int N = 100007, M = 1000007;
int n, m;          //n为待匹配字符串长度
char p[N], s[M];   //p为模板,s搜索范围
int ne[N];         // next数组,初始值为0

int main()
{
    // 第一行输入整数N, 表示字符串P的长度。
    // 第二行输入字符串P。
    // 第三行输入整数M, 表示字符串S的长度。
    // 第四行输入字符串S。
    cin >> n >> p + 1 >> m >> s + 1; //字符串从1开始存储, ci
    n会在末尾加上\0

    // 求next的过程 next[1]=0;从第二个字符开始计算next
    for (int i = 2, j = 0; i <= n; i++) //预处理p串计算前缀
    后缀的值
    {
        //双指针操纵与匹配过程一致
        while (j && p[i] != p[j + 1])
            j = ne[j];
        if (p[i] == p[j + 1])
            j++;
        ne[i] = j; //找到i的回退点j
    }

    // KMP 匹配过程
    for (int si = 1, pj = 0; si <= m; si++) //si,pj为字符串
    指针, 从1开始算
    {
        while (pj && s[si] != p[pj + 1]) // 用p串pj+1字符
        与s串的si比较 (提前看一个字符)
            pj = ne[pj];          //遇到不匹配, 回退p
        j指针
        if (s[si] == p[pj + 1])
            pj++; //pj进一位
        // p串匹配成功
        if (pj == n)
        { //抵达p串的最后一个字符
            cout << si - n << " "; //输出出现位置的下标
            pj = ne[pj];          //回退, 预备下一次的查找
        }
    }
    return 0;
}

```

Python

Python solution

NQ134 模拟堆 AcWing 839

"最后一个数据结构——堆。"小华说, "小根堆: 父节点 $\leq$ 子节点。插入: 放在末尾然后向上冒泡(up)。删除堆顶: 把最后一个元素移到堆顶然后向下沉降(down)。用数组模拟完全二叉树: 节点i的父节点 $=i/2$, 左子 $=2i$, 右子 $=2i+1$ 。堆是优先队列的底层——Dijkstra、Huffman编码、堆排序都离不开它。"

输入

第一行n。然后n行: I x(插入)、PM(输出最小)、DM(删除最小)、D k(删除第k个插入)、C k x(修改第k个插入为x)。

输出

按操作输出。

范围 $1 \leq n \leq 100000$

输入	输出
8	1
I 2	3
I 1	
PM	
DM	
D 1	
C 2 8	
I 3	
PM	

思路 数组模拟小根堆：h[i]存值，size存当前堆大小。down(u)：找u和两子中最小的，若子更小则交换并递归down。up(u)：若u<父节点则交换并递归up。插入：h[++size]=x; up(size)。删除堆顶：h[1]=h[size--]; down(1)。支持随机删除需额外维护ph[k]（第k个插入在堆中位置）和hp[i]（堆位置i对应的插入编号）。

技巧 Python用heapq：heappush/ heappop/ heapify。但竞赛中常需要支持随机删除修改——需要手写堆+位置映射。理解堆的up/down操作是理解所有优先队列变种（d-ary堆、fib堆、配对堆）的基础。数组下标从1开始方便计算父子。

C++

```

#include <iostream>
#include <algorithm>
#include <string.h>

using namespace std;

const int N = 100007;

// hp是heap pointer的缩写，表示堆数组中下标到第k个插入的映射
// ph是pointer heap的缩写，表示第k个插入到堆数组中的下标的映射
// hp和ph数组是互为反函数的
int h[N], ph[N], hp[N], cnt;

void heap_swap(int a, int b)
{
    swap(ph[hp[a]], ph[hp[b]]); // 堆数组的下标
    swap(hp[a], hp[b]); // 下标与第几个插入元素的映射
    swap(h[a], h[b]); // 堆元素
}

void down(int u)
{
    int t = u;
    if (u*2 <= cnt && h[u*2] < h[t]) t = u*2; // u*2左儿子
    if (u*2+1 <= cnt && h[u*2+1] < h[t]) t = u*2+1; // u*2+1右儿子
    if (u != t)
    {
        heap_swap(u, t);
        down(t);
    }
}

void up(int u)
{
    while (u/2 && h[u] < h[u/2])
    {
        heap_swap(u, u/2);
        u >>= 1;
    }
}

int main()
{
    int n, m=0;
    scanf("%d", &n);
    while(n --)
    {
        char op[5];
        int k, x;
        scanf("%s", op);
        // 在堆最后一个元素插入元素
        if (!strcmp(op, "I"))
        {
            scanf("%d", &x);
            cnt ++;
            m++;
            ph[m] = cnt, hp[cnt] = m;
            h[cnt] = x;
            up(cnt); // 插入后执行up(cnt)
        }
        else if (!strcmp(op, "PM")) printf("%d\n", h[1]); // 堆顶是最小值
    }
}

```

Python

Python solution

```

else if (!strcmp(op, "DM"))
{
    heap_swap(1, cnt);
    cnt --;
    down(1); //删除最小值后执行down(1)
}
else if (!strcmp(op, "D"))
{
    scanf("%d", &k); //删除第k个元素
    k = ph[k]; //取第k个元素下标
    heap_swap(k, cnt); //与最后一个元素交换
    cnt--;
    up(k);
    down(k);
}
else {
    scanf("%d%d", &k, &x); //修改第k个数
    k = ph[k];
    h[k] = x;
    up(k);
    down(k);
}
}
return 0;
}

```

本章知识点总结

- 数组模拟数据结构：链表/栈/队列/堆都可以用数组+下标模拟—— $O(1)$ 操作且更缓存友好
- 单调栈/单调队列：利用单调性排除无用元素—— $O(n^2) \rightarrow O(n)$ 的关键技术
- KMP：利用自身前后缀重复——字符串匹配 $O(n+m)$ 。理解失配跳转的智慧
- 堆：完全二叉树+up/down——优先队列 $O(\log n)$ 。是Dijkstra/Huffman/堆排序的基础